

MODELO DE PERSISTÊNCIA CONCEITUAL — EVENT STORE, PROJEÇÕES, STORAGE E RETENÇÃO HISTÓRICA — VERSÃO INICIAL

1. Objetivo do artefato

Este documento define:

- modelo conceitual de persistência;
- organização lógica dos dados;
- estrutura do Event Store;
- projeções materializadas;
- estratégias de retenção;
- persistência temporal;
- arquitetura de armazenamento distribuído.

Seu objetivo é:

- sustentar Event Sourcing;
- permitir replay auditável;
- preservar causalidade histórica;
- organizar persistência multi-temporal;
- suportar escalabilidade institucional.

Este documento representa:

- a fundação persistente da plataforma.

2. Princípios fundamentais

2.1 Persistência é orientada a causalidade

O sistema:

- não persiste apenas estado final;
- persiste evolução histórica do domínio.

2.2 Histórico é ativo institucional

Eventos:

- possuem valor operacional;
- jurídico;
- auditável;
- histórico.

2.3 Persistência deve permitir reconstrução

Toda interpretação:

- deve ser reproduzível;
- a partir do histórico persistido.

2.4 O sistema diferencia armazenamento lógico e físico

Modelo conceitual:

- não depende diretamente da tecnologia física.

O domínio:

- define semântica;
- não implementação específica.

3. Estrutura conceitual da persistência

O modelo é dividido em:

1. Event Store
2. Projeções
3. Read Models
4. Snapshots
5. Storage Operacional
6. Storage Histórico
7. Storage Arquivístico
8. Índices Temporais
9. Metadados
10. Trilhas Auditoriais

4. Camadas de persistência

Camada 1 — Persistência transacional

Responsável por:

- ingestão;
- eventos brutos;
- consistência local.

Camada 2 — Persistência interpretativa

Responsável por:

- projeções;
- consolidações;
- modelos derivados.

Camada 3 — Persistência histórica

Responsável por:

- replay;
- snapshots;
- causalidade histórica.

Camada 4 — Persistência arquivística

Responsável por:

- retenção longa;
- preservação institucional;
- storage frio.

5. Event Store

Objetivo

Persistir:

- fatos históricos imutáveis.

Características obrigatórias

- imutabilidade
- append-only
- versionamento

- ordenação lógica
- replay auditável
- segregação multi-tenant
- rastreabilidade causal

O Event Store NÃO deve

- sobrescrever eventos
- apagar causalidade
- depender de projeções
- armazenar interpretação final como verdade absoluta

6. Estrutura conceitual do evento persistido

Campos mínimos obrigatórios

event_id

event_type

aggregate_id

tenant_id

occurred_at

captured_at

ingested_at

processed_at

schema_version

payload

metadata

correlation_id

causation_id

actor_id

trace_id

Metadados recomendados

- origem
- contexto
- classificação
- criticidade
- vigência normativa
- assinatura
- checksum

7. Estratégia de agregados persistentes

Objetivo

Definir:

- limites de consistência transacional.

Agregados previstos

- JornadaAggregate
- CompetenciaAggregate
- BancoHorasAggregate
- InconsistenciaAggregate
- RegraNormativaAggregate
- HomologacaoAggregate

Regras fundamentais

Agregados devem:

- ser pequenos;
- evitar lock global;
- evitar transações distribuídas.

8. Estratégia de streams

Organização lógica

Streams podem ser segmentadas por:

- tenant
- agregado
- competência
- domínio funcional
- período temporal

Exemplos

`tenant-a.jornada.2026-05`

`tenant-a.bancohoras.usuario-001`

`tenant-a.competencia.2026`

9. Estratégia de particionamento

Objetivo

Permitir:

- escalabilidade;
- replay seletivo;
- isolamento institucional.

Estratégias previstas

Particionamento temporal

Exemplo:

- por competência;
- por ano.

Particionamento institucional

Exemplo:

- por tenant;

- por órgão.

Particionamento funcional

Exemplo:

- auditoria;
- interpretação;
- homologação.

10. Estratégia de projeções

Objetivo

Transformar:

- histórico bruto
em:
- visões consultáveis.

Características

Projeções:

- são derivadas;
- reconstruíveis;
- descartáveis;
- reidratáveis.

11. Tipos de projeção

Operacionais

Exemplo:

- saldo atual;
- jornada diária.

Administrativas

Exemplo:

- pendências;
- homologações.

Temporais

Exemplo:

- timelines;
- reconstruções históricas.

Auditoriais

Exemplo:

- árvore causal;
- replay explicável.

Analíticas

Exemplo:

- indicadores;
- estatísticas;
- métricas institucionais.

12. Read Models

Objetivo

Otimizar:

- leitura;
- pesquisa;
- dashboards;
- APIs operacionais.

Características

Read models:

- não são fonte primária da verdade;
- podem ser regenerados.

13. Estratégia de snapshots

Objetivo

Reduzir:

- custo de replay completo.

Snapshots representam

Estado consolidado:

- em determinado instante temporal.

Gatilhos previstos

- fechamento de competência
- quantidade de eventos
- checkpoints administrativos
- congelamento institucional

Regras fundamentais

Snapshots:

- não substituem eventos;
- apenas aceleram reconstrução.

14. Estrutura conceitual de snapshot

Campos recomendados

snapshot_id

aggregate_id

tenant_id

snapshot_version

created_at

last_event_position

snapshot_payload

checksum

15. Estratégia de reidratação

Objetivo

Reconstruir:

- projeções;
- agregados;
- estados interpretativos.

Processo

snapshot

- replay incremental
- reconstrução do estado
- validação causal

16. Persistência temporal

O sistema deve preservar

- tempo de ocorrência
- tempo de captura
- tempo de ingestão
- tempo de processamento
- tempo administrativo
- vigência normativa

Importância

Essencial para:

- replay;
- auditoria;
- reconstrução histórica.

17. Persistência de regras normativas

Regras devem possuir

- versão
- vigência
- autoria
- histórico completo
- contexto institucional

Regras nunca devem

- sobrescrever vigência anterior;
- destruir interpretação histórica.

18. Persistência auditorial

O sistema deve persistir

- replay executado
- reprocessamentos
- homologações
- mudanças administrativas
- violação de invariantes
- acessos privilegiados

Características

Persistência auditorial:

- possui criticidade extrema.

19. Estratégia multi-tenant

Regras obrigatórias

Persistência deve garantir:

- segregação lógica
- segregação causal
- segregação administrativa
- isolamento auditável

Replay

Nunca pode:

- atravessar tenants.

20. Estratégia de retenção

Camadas de retenção

Quente

Dados:

- recentes;
- altamente acessados.

Morna

Dados:

- históricos consultáveis;
- replay moderado.

Fria

Dados:

- arquivísticos;
- raramente acessados.

Critérios

Retenção depende de:

- criticidade
- contexto jurídico
- necessidade auditável
- frequência de acesso

21. Estratégia de arquivamento

Objetivo

Reduzir:

- custo operacional;
- pressão de storage quente.

Regras

Arquivamento:

- não destrói causalidade;
- não elimina replay;
- preserva auditabilidade.

22. Estratégia de descarte

Nenhum descarte pode

- ser silencioso
- ser irreversível sem auditoria
- eliminar rastreabilidade institucional

Todo descarte deve registrar

- motivo
- autorização
- escopo
- impacto
- trilha causal

23. Estratégia de indexação

Índices previstos

Temporais

Institucionais

Causais

Operacionais

Auditoriais

Objetivos

Permitir:

- replay seletivo;
- busca histórica;

- investigação auditável.

24. Estratégia de integridade

Persistência deve proteger contra

- corrupção histórica
- perda causal
- duplicidade
- inconsistência distribuída
- sobrescrita indevida

Estratégias previstas

- checksums
- hashes
- assinatura
- validação periódica
- trilhas auditáveis

25. Estratégia de consistência

Modelo adotado

Consistência eventual determinística

Características

O sistema admite:

- convergência posterior;
- atualização assíncrona;
- lag controlado.

Mas exige:

- reprodutibilidade histórica.

26. Estratégia de idempotência

Regras obrigatórias

Persistência deve suportar:

- replay repetido
- reentrega de eventos
- duplicidade de mensagens
- reprocessamento seguro

27. Estratégia de segurança

Dados críticos devem possuir

- criptografia
- assinatura
- segregação
- rastreabilidade
- controle de acesso

28. Estratégia de backup e recuperação

O sistema deve suportar

- recuperação total
- recuperação incremental
- restauração temporal
- replay pós-desastre
- reconstrução institucional

29. Estratégia de observabilidade persistente

Persistência deve ser observável

Monitorando:

- crescimento de streams
- falhas de snapshot
- lag de projeção
- corrupção detectada
- throughput de replay

30. Relação com Event Sourcing

Este modelo:

- materializa Event Sourcing;
- sustenta replay;
- preserva causalidade histórica.

31. Relação com CQRS

Read models:

- são derivados do histórico persistido;
- não substituem Event Store.

32. Relação com auditoria

Toda persistência deve permitir:

- rastreamento;
- reconstrução;
- verificação histórica;
- explicabilidade.

33. Relação com governança de dados

Persistência:

- deve obedecer classificação;
- retenção;
- cadeia de custódia;
- compliance institucional.

34. Objetivo arquitetural final

Este modelo busca:

- transformar persistência em mecanismo temporal auditável;
- preservar integridade histórica extrema;
- permitir reconstrução total do domínio;
- sustentar escalabilidade distribuída segura;
- garantir causalidade institucional permanente.